

Guía práctica de demostración

ROS 2 Jazzy con Pixi y CoppeliaSim EDU 4.10 en Windows 11

Procedimiento verificado para instalar ROS 2, crear nodos Python y mover un cubo en CoppeliaSim mediante un topic ROS 2.

Alcance: esta guía incluye únicamente los pasos que funcionaron correctamente durante la práctica. Está pensada para una demostración breve en clase.

1. Esquema general de la demostración

La práctica comprueba una cadena completa de comunicación. Un comando o nodo ROS 2 publica un número real; CoppeliaSim recibe ese número mediante el complemento simROS2; finalmente, un script Lua modifica la coordenada X de un cubo.

PowerShell / nodo Python→	Topic ROS 2	→	simROS2	→	Cubo en CoppeliaSim
Publica Float64	/posicion_cubo		Suscripción Lua		Cambia posición X

2. Instalar Pixi

Objetivo

Instalar el gestor de entornos que utilizaremos para descargar ROS 2 Jazzy y mantener sus dependencias aisladas del resto de programas de Windows.

Dónde se realiza

PowerShell de Windows. Puede abrirse desde el menú Inicio. También sirve la terminal integrada de VS Code si el perfil seleccionado es PowerShell.

Comandos

```
winget install prefix-dev.pixi
```

Explicación

winget es el gestor de paquetes de Windows. La orden descarga e instala Pixi. Después de instalarlo conviene cerrar la terminal y abrir una nueva para que Windows reconozca el comando pixi.

Resultado esperado

Al ejecutar `pixi --version` aparece la versión instalada.

3. Crear el entorno con ROS 2 Jazzy

Objetivo

Crear la carpeta principal del proyecto, inicializar Pixi, añadir los canales de RoboStack y conda-forge e instalar ROS 2 Jazzy Desktop.

Dónde se realiza

PowerShell de Windows.

Comandos

```
mkdir C:\pixi_ws
cd C:\pixi_ws
pixi init
pixi workspace channel add robotstack-jazzy
pixi workspace channel add conda-forge
pixi add ros-jazzy-desktop
pixi shell
ros2 --help
```

Explicación

`mkdir` crea la carpeta `C:\pixi_ws`. `cd` entra en ella. `pixi init` convierte la carpeta en un proyecto Pixi. Los comandos `channel add` indican los repositorios desde los que se descargarán los paquetes. `pixi add ros-jazzy-desktop` instala ROS 2 Jazzy. `pixi shell` abre una sesión con ROS 2 activado. `ros2 --help`

verifica que el comando principal está disponible.

Resultado esperado

La ayuda de ROS 2 se muestra sin errores.

Importante: en las siguientes fases, cuando se indique «dentro de Pixi», la terminal debe haber ejecutado previamente `cd C:\pixi_ws` y `pixi shell`.

4. Crear el workspace y el paquete Python

Objetivo

Crear un workspace de ROS 2 llamado `ros2_ws` y, dentro de él, un paquete Python llamado `robot_ur3`.

Dónde se realiza

PowerShell o terminal integrada de VS Code, dentro de `pixi shell`.

Comandos

```
New-Item -ItemType Directory -Path C:\pixi_ws\ros2_ws\src -Force
cd C:\pixi_ws\ros2_ws\src
ros2 pkg create robot_ur3 --build-type ament_python --license Apache-2.0 --dependencies rclpy
```

Explicación

`New-Item` crea la carpeta `src` del workspace. `ros2 pkg create` genera la estructura del paquete.

`--build-type ament_python` indica que el paquete se programará en Python. `--dependencies rclpy` añade la biblioteca de cliente de ROS 2 para Python.

Resultado esperado

Dentro de `C:\pixi_ws\ros2_ws\src` aparece la carpeta `robot_ur3` con `package.xml`, `setup.py` y el módulo Python.

5. Primer nodo ROS 2 en Python

En VS Code se abrió la carpeta C:\pixi_ws\ros2_ws y se seleccionó como intérprete el Python perteneciente al entorno Pixi. Dentro de src\robot_ur3\robot_ur3 se creó el archivo primer_nodo.py.

Archivo primer_nodo.py

```
import rclpy
from rclpy.node import Node

class PrimerNodo(Node):
    def __init__(self):
        super().__init__('primer_nodo')
        self.get_logger().info('¡Hola desde ROS 2!')

def main(args=None):
    rclpy.init(args=args)
    nodo = PrimerNodo()
    rclpy.spin_once(nodo)
    nodo.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

rclpy.init() inicializa ROS 2; la clase hereda de Node; get_logger().info() escribe un mensaje en consola; rclpy.shutdown() cierra ROS 2 de forma ordenada.

En setup.py se añadió la entrada ejecutable correspondiente dentro de entry_points:

```
entry_points={
    'console_scripts': [
        'primer_nodo = robot_ur3.primer_nodo:main',
    ],
},
```

6. Compilar y ejecutar el nodo

Objetivo

Construir el paquete y comprobar que el nodo se ejecuta.

Dónde se realiza

Terminal integrada de VS Code o PowerShell, dentro de pixi shell.

Comandos

```
cd C:\pixi_ws\ros2_ws
colcon build
.\install\local_setup.ps1
ros2 run robot_ur3 primer_nodo
```

Explicación

colcon build compila el workspace. En esta instalación de Windows se utilizó sin --symlink-install. local_setup.ps1 registra en la terminal actual los paquetes recién compilados. ros2 run ejecuta el nodo.

Resultado esperado

La terminal muestra: «¡Hola desde ROS 2!».

7. Publicador y suscriptor

Se crearon los archivos publicador.py y suscriptor.py para comprobar la comunicación entre dos nodos. Ambos se registraron en setup.py y se añadió `<depend>std_msgs</depend>` en package.xml.

Después de modificar el paquete se volvió a ejecutar colcon build y .install\local_setup.ps1.

Terminal 1:
ros2 run robot_ur3 suscriptor

Terminal 2:
ros2 run robot_ur3 publicador

Comprobaciones:
ros2 node list
ros2 topic list
ros2 topic echo /mensaje_ur3
ros2 topic info /mensaje_ur3

El publicador envía mensajes al topic /mensaje_ur3 y el suscriptor los recibe. Las dos primeras órdenes deben ejecutarse en terminales separadas porque ambos nodos permanecen activos al mismo tiempo.

8. Abrir CoppeliaSim desde el entorno Pixi

Objetivo

Iniciar CoppeliaSim de forma que el complemento simROS2 encuentre las bibliotecas de ROS 2.

Dónde se realiza

PowerShell de Windows, después de entrar en el entorno Pixi.

Comandos

```
cd C:\pixi_ws
pixi shell

Remove-Item Env:QT_PLUGIN_PATH -ErrorAction SilentlyContinue
Remove-Item Env:QT_QPA_PLATFORM_PLUGIN_PATH -ErrorAction SilentlyContinue
Remove-Item Env:QML2_IMPORT_PATH -ErrorAction SilentlyContinue
Remove-Item Env:QML_IMPORT_PATH -ErrorAction SilentlyContinue

$coppelia="C:\Program Files\CoppeliaRobotics\CoppeliaSimEdu"
$env:PATH="$coppelia;$env:PATH"

& "$coppelia\coppeliaSim.exe"
```

Explicación

Las órdenes Remove-Item eliminan variables de Qt que provocaban conflicto entre Pixi y CoppeliaSim. La variable \$coppelia guarda la ruta de instalación. Después, esa ruta se añade temporalmente a PATH y se ejecuta CoppeliaSim.

Resultado esperado

CoppeliaSim se abre sin el error del complemento Qt «windows».

9. Crear la simulación del cubo

Objetivo

Crear una escena mínima con un cuboide que pueda desplazarse en el eje X cuando reciba un valor numérico desde ROS 2.

Dónde se realiza

En la interfaz gráfica de CoppeliaSim EDU 4.10. No se escriben comandos de PowerShell durante esta fase.

Procedimiento

- | | |
|---|---|
| 1 | Crear o abrir una escena vacía en CoppeliaSim. |
| 2 | Insertar un cuboide desde el menú de primitivas de la versión instalada. |
| 3 | Comprobar en el árbol de la escena que el objeto aparece con el alias Cuboid . |
| 4 | Situar el cubo por encima del suelo para que sea visible durante la simulación. |
| 5 | Seleccionar el cubo en el árbol de la escena. |
| 6 | Añadirle un Simulation Script de tipo Non threaded y lenguaje Lua . |
| 7 | Abrir el editor del script, borrar el contenido inicial y pegar el código del apartado siguiente. |
| 8 | Guardar la escena y pulsar el botón de inicio de la simulación. |

Importante: el nombre utilizado por el script debe coincidir con el alias del objeto. En la escena que funcionó correctamente el objeto se llamaba /Cuboid. Otra opción válida, si el script está asociado directamente al cubo, es obtener el objeto relativo mediante `sim.getObject('..')`.

10. Script Lua del cubo

El script se suscribe al topic /posicion_cubo. Cuando llega un mensaje `std_msgs/msg/Float64`, la función `posicionRecibida` conserva Y y Z y sustituye únicamente la coordenada X.

```
function posicionRecibida(msg)
    local posicion = sim.getObjectPosition(cubo, sim.handle_world)
    posicion[1] = msg.data
    sim.setObjectPosition(cubo, posicion, sim.handle_world)

    sim.addLog(
        sim.verbosity_scriptinfos,
        string.format('Nueva posición X recibida: %.3f m', msg.data)
    )
end

function sysCall_init()
    sim = require('sim')
    simROS2 = require('simROS2')

    cubo = sim.getObject('/Cuboid')

    suscriptor = simROS2.createSubscription(
        '/posicion_cubo',
        'std_msgs/msg/Float64',
        'posicionRecibida'
    )

    sim.addLog(
        sim.verbosity_scriptinfos,
        'Esperando posiciones en /posicion_cubo'
    )
end

function sysCall_cleanup()
    if suscriptor then
        simROS2.shutdownSubscription(suscriptor)
    end
end
```

Explicación del script

Elemento	Función
<code>sim = require('sim')</code>	Carga la API principal de CoppeliaSim.
<code>simROS2 = require('simROS2')</code>	Carga el complemento que permite crear publicadores y suscriptores ROS 2.
<code>sim.getObject('/Cuboid')</code>	Obtiene el identificador interno del cuboide por su alias.
<code>createSubscription(...)</code>	Crea la suscripción al topic /posicion_cubo con mensajes Float64.
<code>posicionRecibida(msg)</code>	Se ejecuta automáticamente cada vez que llega un mensaje.
<code>posicion[1] = msg.data</code>	Sustituye la coordenada X; en Lua los vectores empiezan en el índice 1.
<code>sim.setObjectPosition(...)</code>	Aplica la nueva posición respecto al sistema de coordenadas mundial.
<code>sysCall_cleanup()</code>	Elimina correctamente la suscripción al detener la simulación.

Al iniciar la simulación, la consola de CoppeliaSim debe mostrar: Esperando posiciones en /posicion_cubo. Este mensaje confirma que el script se ejecuta, que simROS2 se ha cargado y que la suscripción ha sido creada.

11. Enviar posiciones al cubo

Objetivo

Publicar valores numéricos desde ROS 2 y observar cómo el cubo cambia de posición.

Dónde se realiza

Una terminal nueva de PowerShell o de VS Code. Coppeliasim debe continuar abierto y la simulación debe estar en marcha. En la terminal hay que activar Pixi y cargar el workspace.

Comandos

```
cd C:\pixi_ws
pixi shell

cd C:\pixi_ws\ros2_ws
.\install\local_setup.ps1

ros2 topic pub --once /posicion_cubo std_msgs/msg/Float64 "{data: 0.5}"
ros2 topic pub --once /posicion_cubo std_msgs/msg/Float64 "{data: -0.5}"
ros2 topic pub --once /posicion_cubo std_msgs/msg/Float64 "{data: 0.0}"
```

Explicación

--once hace que se publique un único mensaje y el comando termine. /posicion_cubo es el nombre del topic. std_msgs/msg/Float64 indica el tipo del mensaje. El campo data contiene la nueva coordenada X en metros.

Resultado esperado

El cubo se desplaza primero a $X = 0,5$ m, después a $X = -0,5$ m y finalmente vuelve a $X = 0$ m. La consola de Coppeliasim muestra cada valor recibido.

12. Comprobaciones útiles

Mientras la simulación está ejecutándose se pueden emplear las siguientes órdenes desde otra terminal activada con Pixi:

```
ros2 topic list
ros2 topic info /posicion_cubo
ros2 topic echo /posicion_cubo
```

topic list muestra los topics existentes. topic info permite comprobar que existe un suscriptor. topic echo muestra los mensajes que se publican.

13. Resultado final

La demostración valida correctamente toda la cadena:

- ✓ ROS 2 Jazzy instalado en Windows mediante Pixi.
- ✓ Paquete Python compilado con colcon.
- ✓ Nodos ROS 2 ejecutados desde VS Code o PowerShell.
- ✓ Comunicación entre publicador y suscriptor.
- ✓ Coppeliasim iniciado dentro del entorno ROS 2.
- ✓ Complemento simROS2 cargado desde Lua.
- ✓ Movimiento de un objeto de Coppeliasim mediante un topic ROS 2.

A partir de esta base, el siguiente paso puede ser sustituir el cubo por una articulación del UR3 y publicar un ángulo objetivo desde ROS 2.